

Transformer Network (Part 2)

Introduction to Sequential Data Processing

Kyuhong Shim

Department of Intelligent Software
Sungkyunkwan University

2026. 4. 6.

- **Self-attention** is an adaptive weighted sum over a sequence, and is a core mechanism for time-series data processing.
- However, self-attention alone is not yet a **full Transformer architecture**.
- A practical Transformer needs:
 - ▶ **Residual connections**
 - ▶ **Layer normalization**
 - ▶ **Position-wise feed-forward networks**
 - ▶ **Stacking of multiple blocks**

- **Transformer block** contains two main sublayers:
 1. **Multi-Head Self-Attention (MHSA)**
 2. **Feed-Forward Network (FFN)**
- A standard block applies:
 1. MHSA to exchange information across timesteps
 2. FFN to transform each timestep independently
 - ▶ By stacking multiple Transformer blocks, we can build a deep Transformer network.
 - ▶ The Transformer block repeatedly performs *context mixing* (MHSA) $\rightarrow$ *nonlinear transformation* (FFN).

- Around the sublayers, we add:
 - ▶ **Residual connections**
 - ▶ **Layer normalization (LN)**
- Let the input to one block be

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_0^\top \\ \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_{T-1}^\top \end{bmatrix} \in \mathbb{R}^{T \times d}.$$

- The input and output shapes are usually the same:

$$\mathbf{X} \in \mathbb{R}^{T \times d} \longrightarrow \mathbf{Y} \in \mathbb{R}^{T \times d}.$$

- **Residual connection** adds the input of a sublayer back to its output:

$$\mathbf{y} = \mathbf{x} + f(\mathbf{x})$$

- ▶ $f(\cdot)$ can be an MHSA or FFN sublayer.
- Intuition:
 - ▶ Each sublayer does not need to build a new representation from scratch, and only needs to learn a useful refinement to the input.
 - ▶ Residual connection improves gradient flow, optimization stability, and information preservation across layers.

- **Layer normalization (LN)** normalizes each feature vector across its dimension.
- For a vector $\mathbf{x} \in \mathbb{R}^d$,

$$\text{LN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \mu(\mathbf{x})}{\sigma(\mathbf{x})} + \beta, \quad \gamma, \beta \in \mathbb{R}^d.$$

- ▶ $\mu(\mathbf{x}) = \frac{1}{d} \mathbf{1}^\top \mathbf{x}$
 - ▶ $\sigma(\mathbf{x}) = \sqrt{\frac{1}{d} \|\mathbf{x} - \mu(\mathbf{x}) \mathbf{1}\|^2 + \varepsilon}$
 - ▶ $\mu(\mathbf{x}) \in \mathbb{R}$ and $\sigma(\mathbf{x}) \in \mathbb{R}$.
 - ▶ For simplicity, assume vector-scalar operations are *broadcasted* elementwise.
- LN is applied independently at every timestep.

- As features (i.e., vectors) pass through many layers, their scale may drift.
- Then, optimization becomes harder because different layers may operate in very different ranges.
- Layer normalization helps optimization by:
 - ▶ Stabilizing feature magnitudes and feature statistics
 - ▶ Keeping pre-affine features better standardized
 - ▶ Reducing sensitivity to internal scale changes
- Unlike batch normalization, layer normalization does **not** rely on batch statistics.

- Ignore the affine parameters γ, β and define $\hat{\mathbf{x}}$ as:

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu(\mathbf{x})}{\sigma(\mathbf{x})}$$

- Then $\hat{\mathbf{x}}$ always satisfies two constraints (assume $\epsilon = 0$):

$$\mathbf{1}^\top \hat{\mathbf{x}} = 0 \quad (\text{zero-mean hyperplane})$$

$$\frac{1}{d} \|\hat{\mathbf{x}}\|^2 = 1 \quad (\text{fixed-radius hypersphere})$$

- Therefore, LN removes two degrees of freedom: *global shift* and *global scale*.
 - ▶ γ and β try to compensate these constraints, but the effective *rank* is reduced after applying LN. This is called rank-collapsing behavior.

- LN removes the gradient components along:
 - ▶ All-ones direction $\mathbf{1}$ (shift direction)
 - ▶ Centered input direction $\mathbf{c} = \mathbf{x} - \mu(\mathbf{x})$ (scale direction)
- This means that some directions are always *annihilated* after LN.
 - ▶ Repeated LN can weaken gradients in certain directions.
 - ▶ This is one reason why residual connections are important to preserve gradient flow.

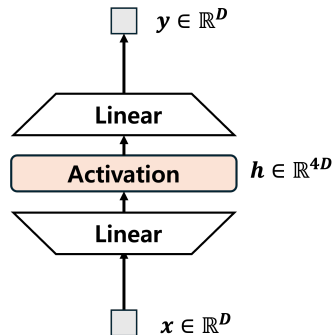
- **Feed-forward network (FFN)** applies nonlinear transform to each feature vector.

$$\text{FFN}(\mathbf{x}_t) = \mathbf{W}_2 \phi(\mathbf{W}_1 \mathbf{x}_t + \mathbf{b}_1) + \mathbf{b}_2.$$

- Usually, FFN first expands the dimension and then projects it back:

$$\mathbf{W}_1 \in \mathbb{R}^{d \times d_{\text{ff}}}, \quad \mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d}, \quad d_{\text{ff}} \gg d$$

- ▶ A common convention is to use $d_{\text{ff}} = 4 \times d$.
- ▶ ϕ is a nonlinear activation function, such as ReLU, Swish, or GELU.



- Recall the matrix form of self-attention:

$$\mathbf{Y} = \mathbf{A}\mathbf{V}, \quad \mathbf{A} = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d}} + \mathbf{M} \right)$$

- Once the attention matrix $\mathbf{A}$ is computed, the aggregation step is a linear weighted sum of value vectors over time.

$$\mathbf{V} \longrightarrow \mathbf{A}\mathbf{V}$$

- Different roles of SA and FFN:
 - MHSA performs **adaptive token mixing**.
 - FFN adds pointwise (i.e., per-step) **nonlinearity and feature mixing**.
 - Without FFN, the Transformer block would be much less expressive.

- The original Transformer block is designed as a sequence of two sublayers:

$$\mathbf{H}' = \text{LN}(\mathbf{X} + \text{MHSA}(\mathbf{X}))$$

$$\mathbf{Y} = \text{LN}(\mathbf{H}' + \text{FFN}(\mathbf{H}'))$$

- Interpretation:
 - ▶ MHSA adds contextual information from the sequence.
 - ▶ FFN transforms each contextualized representation.
- This block maps $\mathbf{X} \in \mathbb{R}^{T \times d}$ to $\mathbf{Y} \in \mathbb{R}^{T \times d}$.

- We stack multiple Transformer blocks:

$$\mathbf{X}^{(\ell+1)} = \text{Block}^{(\ell)}(\mathbf{X}^{(\ell)}), \quad \ell = 0, 1, \dots, L-1.$$

- ▶ $\mathbf{X}^{(0)}$ is the input representation
 - ▶ $\mathbf{X}^{(L)}$ is the final hidden representation
 - ▶ Each layer has its own trainable parameters
- Advantages of layer stacking:
 - ▶ Although a single SA layer can see the entire sequence, a deeper model can generally learn more complex temporal interactions.
 - ▶ We expect deeper (i.e., upper) layers to learn more abstract and task-dependent representations.

- Transformer is a **deep sequence model**:

$$(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{T-1}) \longrightarrow (\mathbf{h}_0^{(L)}, \mathbf{h}_1^{(L)}, \dots, \mathbf{h}_{T-1}^{(L)})$$

- After stacking many Transformer blocks, each timestep representation contains information from a broad **context**.
- Each output vector is a **contextualized** representation:
 - ▶ Still corresponds to a specific position
 - ▶ But also reflects information from other positions

- In the **post-norm** design, LN is applied **after** each residual addition:

$$\mathbf{H}' = \text{LN}(\mathbf{X} + \text{MHSA}(\mathbf{X}))$$

$$\mathbf{Y} = \text{LN}(\mathbf{H}' + \text{FFN}(\mathbf{H}')) .$$

- ▶ This is the original Transformer formulation.

- In the **pre-norm** design, LN is applied **before** each sublayer:

$$\mathbf{H}' = \mathbf{X} + \text{MHSA}(\text{LN}(\mathbf{X}))$$

$$\mathbf{Y} = \mathbf{H}' + \text{FFN}(\text{LN}(\mathbf{H}')) .$$

- ▶ This version is often more stable when stacking many layers.
- ▶ Most modern Transformers adopt pre-norm architecture.

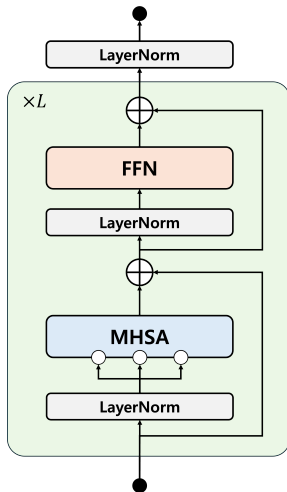
- In practice, pre-norm Transformers often add an additional LN at the end of the stack:

$$\mathbf{x}^{(\ell+1)} = \text{Block}^{(\ell)}(\mathbf{x}^{(\ell)}), \quad \ell = 0, \dots, L-1$$

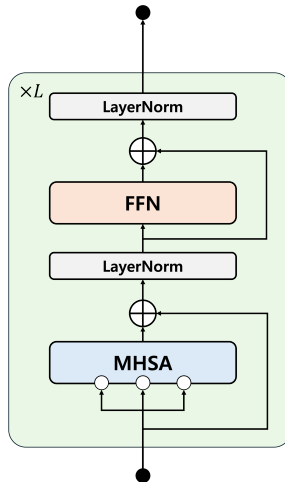
$$\mathbf{H}_{\text{final}} = \text{LN}(\mathbf{x}^{(L)}).$$

- Intuition:
 - ▶ Internal LNs stabilize each block.
 - ▶ The final LN also stabilizes the output features used by the output head or other modules.
- Pre-norm is common in modern decoder-only Transformers (will discuss later).

Pre-Norm vs. Post-Norm



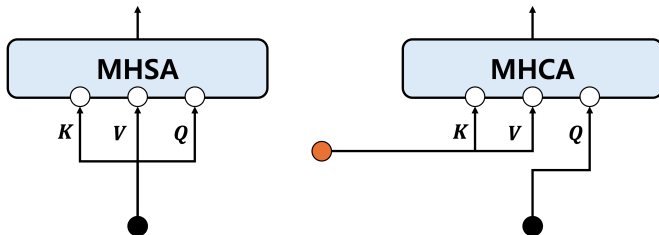
Pre-norm Transformer



Post-norm Transformer

- Self-attention mixes the information gathered from the *same* sequence.
- **Cross-attention (CA)** attends to the **source** sequence, which is different from the current **target** sequence.
 - ▶ Target sequence provides queries, and the source sequence provides keys and values.
 - ▶ CA aggregates information from the *source* sequence.

$$q_t = W_Q x_t^{\text{tgt}}, \quad k_s = W_K x_s^{\text{src}}, \quad v_s = W_V x_s^{\text{src}}$$



- The target timestep t scores each source timestep s by

$$e_{t,s} = \frac{\mathbf{q}_t^\top \mathbf{k}_s}{\sqrt{d}}, \quad \alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_j \exp(e_{t,j})}$$

- The output at target timestep t is

$$\mathbf{y}_t = \sum_s \alpha_{t,s} \mathbf{v}_s$$

- Cross-attention is essentially the same mechanism as self-attention:
 - ▶ Target timestep asks, “Which source positions are relevant to me?”
 - ▶ Cross-attention gathers the relevant information from the source sequence.
 - ▶ Similarly, we can design a multi-head cross-attention (MHCA).

- The length of the source and target can be different.
- Stack all target queries and all source keys/values:

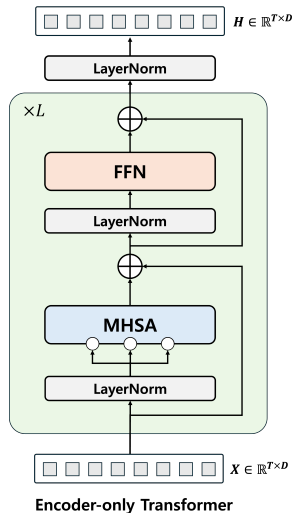
$$\mathbf{Q}_{\text{tgt}} \in \mathbb{R}^{T_{\text{tgt}} \times d}, \quad \mathbf{K}_{\text{src}}, \mathbf{V}_{\text{src}} \in \mathbb{R}^{T_{\text{src}} \times d}$$

- Cross-attention can be written as:

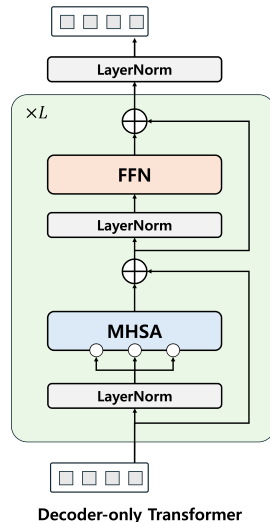
$$\text{CrossAttention}(\mathbf{Q}_{\text{tgt}}, \mathbf{K}_{\text{src}}, \mathbf{V}_{\text{src}}) = \text{softmax} \left(\frac{\mathbf{Q}_{\text{tgt}} \mathbf{K}_{\text{src}}^{\top}}{\sqrt{d}} + \mathbf{M} \right) \mathbf{V}_{\text{src}}.$$

- ▶ The attention matrix is now in $\mathbb{R}^{T_{\text{tgt}} \times T_{\text{src}}}$.
- ▶ The **mask** $\mathbf{M}$ can also be added in the equation, indicating which source timesteps are visible (i.e., can be attended to) or not in perspective of the target.

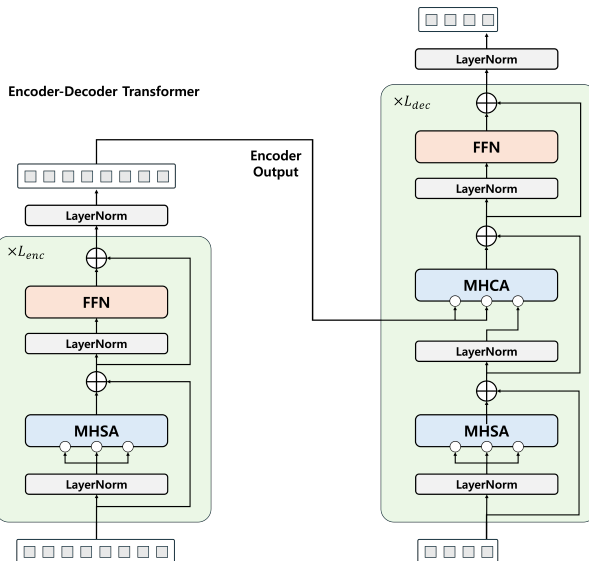
- **Encoder-only Transformer** uses **non-causal self-attention**.
 - ▶ Each position can see the entire input sequence.
 - ▶ The entire sequence is available.
- Applications:
 - ▶ Sequence retrieval
 - ▶ Sequence classification
 - ▶ Representation learning (feature extraction)



- **Decoder-only Transformer** uses **causal self-attention**.
 - ▶ Position t can only attend to positions $0, 1, \dots, t$.
 - ▶ It is naturally suited for **autoregressive generation**.
- Applications:
 - ▶ Language generation
 - ▶ Future prediction (forecasting)
 - ▶ Causal language modeling



- **Encoder-Decoder Transformer** contains:
 - ▶ A Transformer encoder that processes the **source sequence**.
 - ▶ A Transformer decoder that generates the **target sequence**.
- If the task maps one source sequence to a different target sequence, the encoder-decoder architecture is a natural choice.
 - ▶ The source sequence is already given.
 - ▶ We need to generate the target sequence based on the source sequence.
- Applications:
 - ▶ Machine translation
 - ▶ Summarization
 - ▶ Image captioning and speech recognition



- In sequential modeling, a common objective is **next-step prediction**.
- Given the past observations, the model predicts the next one autoregressively:

$$\hat{\mathbf{x}}_{t+1} = f_{\theta}(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_t)$$

- In probabilistic form, this can be written as:

$$\underset{\theta}{\text{maximize}} \quad p(\mathbf{x}_{t+1} \mid \mathbf{x}_{\leq t})$$

- The total loss is the sum of losses in each timestep:

$$\mathcal{L} = - \sum_{t=0}^{T-2} \log p(\mathbf{x}_{t+1} \mid \mathbf{x}_{\leq t}) = - \log \prod_{t=0}^{T-2} p(\mathbf{x}_{t+1} \mid \mathbf{x}_{\leq t}) = \underbrace{-\log p(\mathbf{X})}_{\text{negative log-likelihood}}$$

- During training, we use **teacher forcing**.
 - ▶ Instead of feeding the model's own previous prediction, we feed the **ground-truth previous element**.
 - ▶ We already know the full target sequence during training.
 - ▶ Teacher forcing makes optimization easier because the model always conditions on the correct past context during training.
- Input-label formulation for training:
 - ▶ Input: $(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{T-2})$
 - ▶ Label: $(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{T-1})$

- **Training**

- ▶ The full sequence is available
- ▶ Causal masking enforces the autoregressive constraint
- ▶ Losses for all positions can be computed in parallel

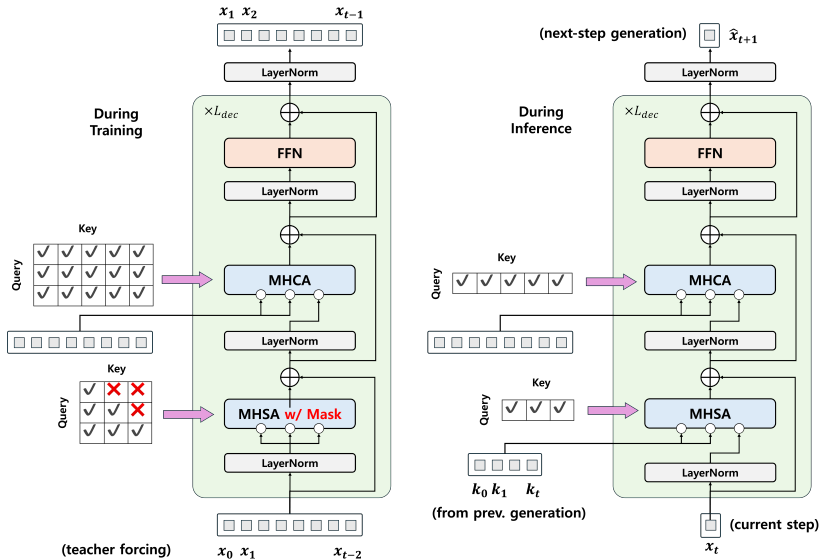
- **Inference**

- ▶ Future ground truth is not available
- ▶ Must generate one step at a time

- Transformer decoder is highly parallel during training, but not fully parallel during autoregressive generation.

- ▶ Teacher forcing leverages the fact that all positions can be trained in parallel, which is one key advantage of the Transformer network.
- ▶ However, this causes a train-inference mismatch, often called **exposure bias**.

Parallel Training vs. Sequential Inference



- **Adaptive contextual aggregation**
 - ▶ Each position can dynamically weight other positions
 - ▶ Global receptive field over the visible sequence
- **Strong representation power**
 - ▶ Self-attention + FFN + depth gives rich nonlinear modeling capacity
 - ▶ Theoretically, many sequential relationships can be modeled by the Transformer.
- **Highly parallel training**
 - ▶ All positions can be processed together during training.
- **Flexible architecture**
 - ▶ Encoder-only, Decoder-only, Encoder-Decoder
 - ▶ Choose an appropriate design for different applications

- **Quadratic attention cost**
 - ▶ Self-attention scales as $\mathcal{O}(T^2)$ in sequence length
- **Sequential autoregressive inference**
 - ▶ Generation still proceeds one step at a time
- **Dependence on positional design**
 - ▶ Attention alone does not encode order
 - ▶ Requires an additional positional encoding module